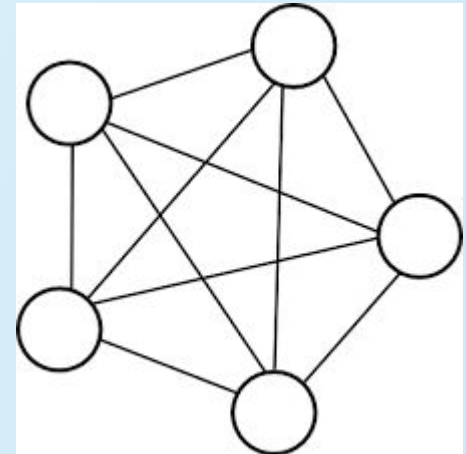


Introduction aux Graphes et Optimisations

Modéliser les relations et trouver la meilleure solution possible.

Réalisée par:

Ben Béchir Chaima





1

Qu'est-ce qu'un Graphe ?

2

Types de Graphes et Représentation

3

Algorithmes de Parcours

4

**Algorithmes du Plus Court Chemin (Dijkstra,
Bellman-Ford)**

5

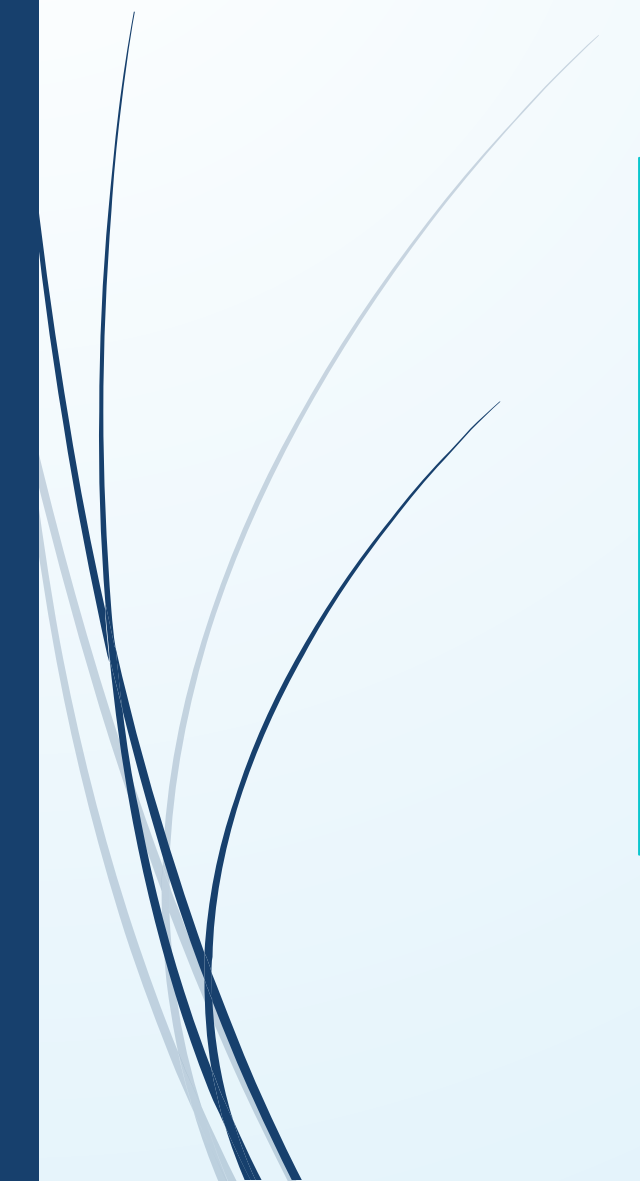
Problème du Flot Maximal et Applications

6

Introduction à l'Optimisation (Programmation Linéaire)

7

Conclusion et Applications



Qu'est-ce qu'un Graphe ?

Définition et Composants

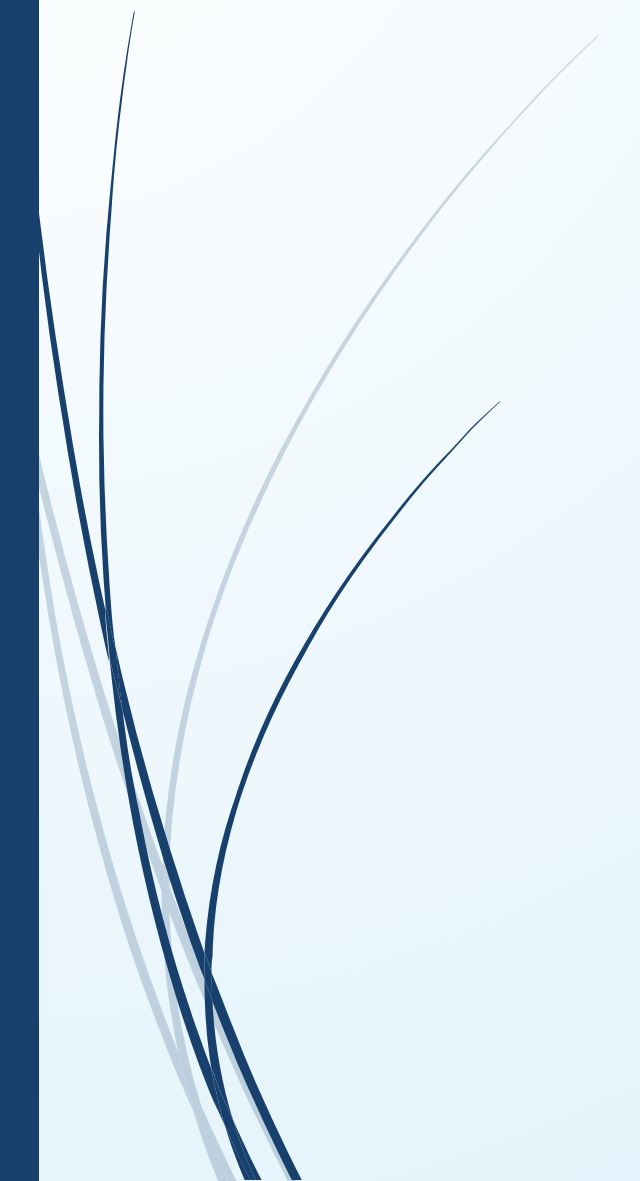
Section	Contenu
Définition	Un Graphe est une structure mathématique utilisée pour modéliser des relations entre des objets. Il est défini par un ensemble de points et un ensemble de liens.
Nœuds (Sommets)	Les objets ou entités (points) que l'on modélise.
Arêtes (Arcs)	Les liens ou relations entre les nœuds.
Analogie	Une carte routière : les villes sont les Nœuds, et les routes sont les Arêtes. Les Réseaux Sociaux : les utilisateurs sont les Nœuds, les amitiés sont les Arêtes.
But	Permettre aux algorithmes d'analyser, de quantifier et d'optimiser les relations (chemins, flux, dépendances) au sein d'un système.



Types de Graphes et Représentation

Classifications

Type de Graphe	Caractéristique	Représentation Algorithmique
Non Orienté	Les arêtes n'ont pas de direction (Alié à B implique B lié à A).	Liste d'Adjacence : Pour chaque nœud, lister ses voisins. (Espace-mémoire efficace).
Orienté (Digraphe)	Les arêtes ont une direction (Ex : A envoie vers B, mais B n'envoie pas forcément vers A).	Matrice d'Adjacence : Tableau carré où la valeur $M[i][j]$ indique s'il existe un arc de i à j. (Rapide pour vérifier les connexions).
Pondéré	Chaque arête possède une valeur ou un poids (coût, distance, temps).	Le poids est stocké dans la Liste ou la Matrice. Essentiel pour les problèmes d'optimisation.
Cyclique	Contient au moins un chemin qui permet de revenir au nœud de départ.	
Acyclique	Ne contient aucun cycle. (DAG : Directed Acyclic Graph, très utilisé pour les dépendances de tâches).	



Algorithmes de Parcours

Explorer le Graphe

Algorithme	Acronyme	Principe d'Exploration	Application Typique
Parcours en Largeur	BFS (Breadth-First Search)	Explore le graphe par niveaux (de proche en proche). Utilise une file (Queue).	Trouver le chemin le plus court en nombre d'arêtes.
Parcours en Profondeur	DFS (Depth-First Search)	Explore le graphe en allant aussi loin que possible dans une branche avant de remonter. Utilise une pile (Stack).	Détection de cycles, tri topologique.

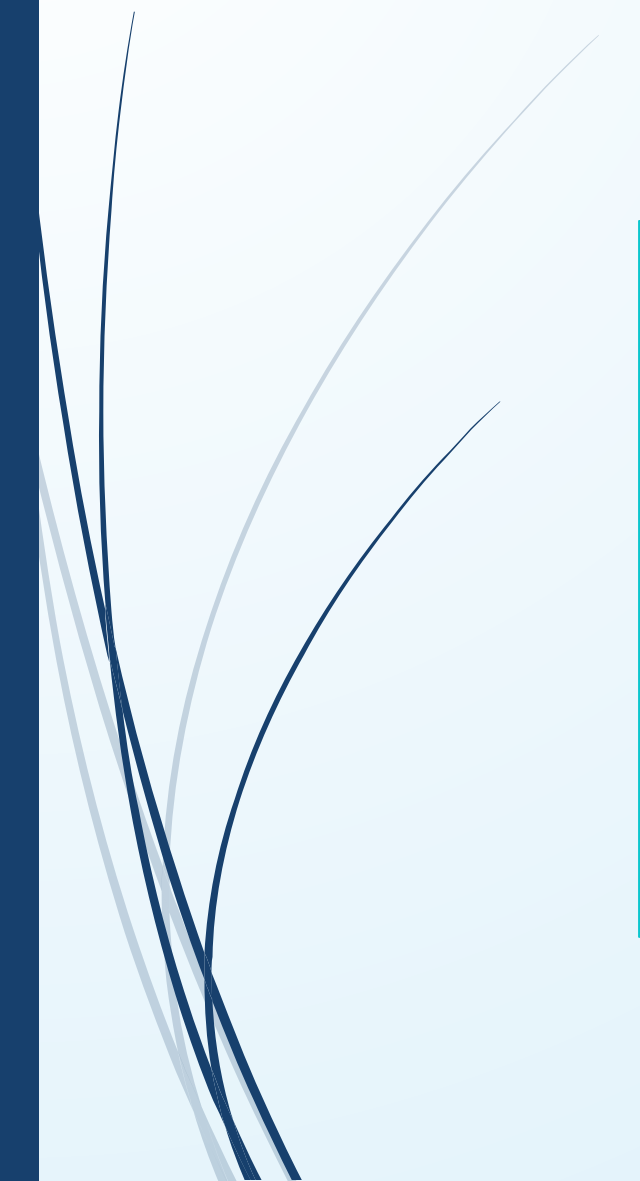


Algorithmes du Plus Court Chemin

Optimisation de Route

Algorithmes
du Plus Court
Chemin

Algorithme	Objectif	Contraintes et Caractéristiques
Dijkstra	Trouver le plus court chemin entre un nœud source unique et tous les autres nœuds du graphe.	Nécessite des poids d'arêtes positifs ou nuls. Le plus utilisé en pratique (GPS, routage réseau).
Bellman-Ford	Trouver le plus court chemin entre un nœud source unique et tous les autres nœuds.	Peut gérer des poids d'arêtes négatifs. Permet de détecter l'existence de cycles négatifs (ce que Dijkstra ne peut pas faire).
Floyd-Warshall	Trouver le plus court chemin entre toutes les paires de nœuds dans le graphe.	Utilisé lorsque l'on a besoin de connaître tous les chemins optimaux possibles.



Problème du Flot Maximal

Capacité et Flux

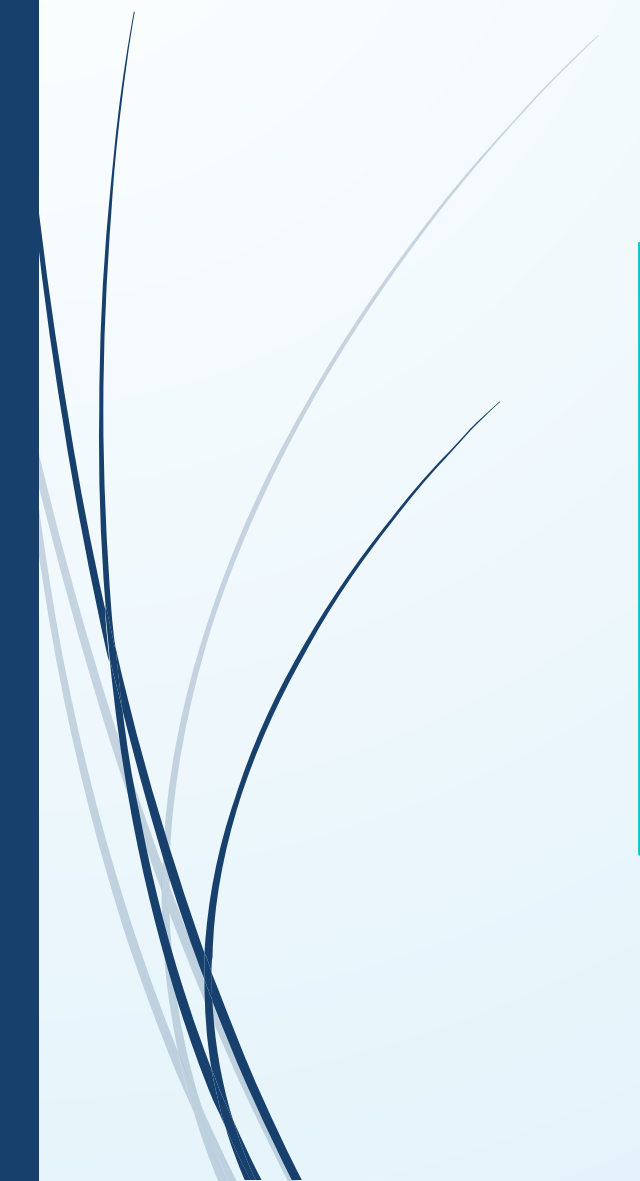
Concept	Objectif	Règle Clé
Réseau de Flot	Graphe orienté et pondéré où les poids des arêtes représentent des capacités (quantité maximale pouvant passer par ce lien).	Modélisation des réseaux de distribution (eau, électricité, paquets de données, trafic routier).
Source/Puits	La Source (S) est le nœud d'où part le flot ; le Puits (T) est le nœud où le flot se termine.	
Flot Maximal	La quantité totale maximale de "matière" qui peut être envoyée de la Source au Puits sans dépasser la capacité d'aucune arête.	Optimisation du trafic réseau pour éviter la congestion.
Algorithme Clé	Ford-Fulkerson ou Edmonds-Karp. Ils augmentent itérativement le flot le long des chemins disponibles jusqu'à ce qu'un état d'équilibre soit atteint.	



Introduction à l'Optimisation

Programmation Linéaire

Concept	Description	Exemple
Optimisation	Domaine visant à choisir le meilleur élément parmi un ensemble d'alternatives disponibles.	Minimiser les coûts, maximiser les profits, réduire le temps de transport.
Fonction Objectif	L'équation que l'on cherche à maximiser (ex: $Z = 2x + 3y$) ou à minimiser.	
Contraintes	Les limites ou conditions qui restreignent les solutions possibles (ex: Budget, Temps, Capacité physique).	
Programmation Linéaire	Technique d'optimisation où la fonction objectif et toutes les contraintes sont des relations linéaires.	Utilisé dans la gestion des ressources, la planification de la production.



Conclusion

En résumé

Conclusion

Points clés	Contenu
Essentiel	Les graphes sont le langage formel pour modéliser les relations et les systèmes interconnectés.
Algorithmes Clés	Les algorithmes comme Dijkstra (plus court chemin) et Ford-Fulkerson (flot maximal) permettent de résoudre des problèmes d'optimisation complexes.
Optimisation	Le but est de trouver la meilleure solution possible dans un espace de contraintes donné.
Message final	Des réseaux sociaux au GPS, de la logistique à la planification de production, l'analyse par les graphes et l'optimisation sont au cœur des systèmes de prise de décision modernes.

Merci de votre
attention!

